## All the tasks that I have shared in our whatsapp group

### 1. DATAWAREHOUSE TASKS:

Team,
I have uploaded the upto date ppts in our gdrive and webpage also, Please try to complete the following tasks before we meet tomorrow and get prepared for the quick quiz of 12 questions on the below points(1,2,3).
https://drive.google.com/drive/folders/1nK-DQK_M2QBvqHsFgvNQWRxj1Jk5lqhc?usp=drive_link
1. Simply go through & recognize the terminologies/abbrevations from 1_Inceptez_BIGDATA_Upto_Speed_WE49.pptx Slide 50 to 59.
2. Go through slide 8 to 11 in project1_datawarehouse_WE49.pptx
3. Go through the upgraded version of https://inceptezlabs.com/bigdata/ (refresh the page and try, you should see the color menus, otherwise press ctrl+shift+del and cached images & files -> delete data), go through menu - all about bigdata & data architecture overview & datawarehouse deepdive.
4. Follow slide 9 & 10 Just create a github repository (task1) for your future need (I will explain it in detail soon on how to push/pull code, just do only registration & repository creation alone for now).
5. Follow slide 11 to do (task2) to create 'Conceptual data model' of legacy DWH.

6. Can you go through slide 5 (business objectives) and create a flowchart referring slide 11 example for flowchart creation?
7. After completing the task 4, fill your git repo info alone in the below gsheet https://docs.google.com/spreadsheets/d/1MAu50LJzUvwUqJ4frtfNjJGWceXz14gvbu5BkYLYLmU/edit?usp=sharing
8. Follow the attached image and FOLLOW the git repository of our other aspirants.
9. I have modified our quiz tool to show the score also in the last, pls try the quiz once again taking your own time
10. Refer slide 12 example and can you create an erDiagram with the below scenario and push/upload the erdiagram into your git repository.
Employee & Department ER Scenarios
One Employee may have zero or one Manager.
→ 1 : 0..1
One Employee may got on zero or many Promotions
→ 1 : 0..Many
One Employee belongs to exactly one Department.
→ 1 : 1
One Department can have one or many Employees.
→ 1 : 1..Many
One Employee can work on one or many Projects, and one Project can have one or many Employees.
→ 1..Many : 1..Many

11. Refer the 'Mermaid DFD Diagram Sample Code' and try to build the DFD diagram for the entities of EMPLOYEES, BRANCHES & CUST_PROFILE for the layers of Source to Staging to DSA to ODS.
12. Refer the 'Physical Data Model - Data Definition Language Script (DDL)' and customize the schema name adding yourname as a suffix then use the mysql workbench to create stgdb_yourname, tables inside the DSA(Staging) database and create odsdb_yourname, tables inside the ODS database.
13. Follow the video/steps & install Python in your Windows/Mac.
https://inceptezlabs.com/bigdata/ -> Projects -> BFSILegacyDWProject1 -> Prerequisite Setup -> Programming / ETL Code
14. Go through & understand under 'Development — End to End Implementation' -> Mapping Document (Source → Transform → Target)
15. Go through & understand under 'How this Python script works — Step by Step' -> click on the > step 1 to 6 and read through the way how python program works.
16. In your Windows/Mac PC open the command prompt
Search -> cmd -> python
17. Run the below program to load data into one table (Branches)

```
#a. Import Libraries
import pandas as pd
from sqlalchemy import create_engine

#b. Connection Credentials
#Change these to match your MySQL server details
username  = 'inceptez'
password  = "Inceptez%40123"  # %40 is URL-encoded @
host      = '34.174.250.128'
db        = 'stgdb_mkirfan'#Change this to your schema
table     = 'stg_branches'

#c. Create DB Engine (pymysql driver)
engine = create_engine(
    f"mysql+pymysql://{username}:{password}@{host}:3306/{db}")

#d. Source Folder Path
#Change this to where you uncompressed the dataset zip
folder =
"E:\\2026_BATCHES\\inceptezlearn\\dwhproject\\source_landing_pad\\bfsi_dataset\\branches.csv"

#e. Load the data from the file to pandas dataframe to do structured queries
df = pd.read_csv(folder)
```

#f. Write the dataframe data into the table (stg_branches) using the engine, without index value in the first column & replace/overwrite the table data.
df.to_sql(table, con=engine, index=False, if_exists="replace")
print(f"Rows loaded into table: {table}")

18. Task18: Goto our portal https://inceptezlabs.com/bigdata/ and go through the 'Development — End to End Implementation' -> 'Creation of Mapping Documents' and understand how a mapping document looks like and how ETL rules are provided to developers by a lead/architects.
19. Task19: Explore 'Staging Layer Load', download the data into your windows drive, execute 'Staging → DSA Load' (If you already completed the python load on one table, run the below code by changing the 'Python — stg_load.py' code copied from the portal, ensure to change the following credentials
username  = 'testusr' # Change to inceptez
password  = "Inceptez%40123"  # %40 is URL-encoded @
host      = '34.63.107.222' # Change to 34.174.250.128
db        = 'stgdb' # Change to your dbname

folder = "D:\\Downloads\\bfsi_dataset\\" # Change to your folder name for eg. E:\2026_BATCHES\inceptezlearn\dwhproject\source_landing_pad\bfsi_dataset\\

20. Task20: DSA to ODS (Operational Data Store) Layer Load
Execute the 'SQL — ods_load.sql' task in the portal to load data from DSA to ODS.

21. Task21: ODS to DWH Load
Execute the 'SQL — 4_load_dwh_dim_fact_aggr.sql' task in the portal to load data from ODS to DWH. Just try only this load - # SCD Type1 - History is not maintained, We will see '# SCD Type 2 Starts here' on next weekend..

**#SCD type1 dim load**
create table learning.ods_cust_src(id int,name varchar(100),finstatus varchar(100),address varchar(100));
#Source send the data in a complete fashion, not incremental fashion
insert into learning.ods_cust_src (id,name,address) values(1,'sojiv','1,xyz street, area,city');#day1
insert into learning.ods_cust_src (id,name,address) values(2,'rizwan','2,abc street, area,city');#day2
insert into learning.ods_cust_src (id,name,address) values(3,'sujithra','2,abc street, area,city');#day3
update learning.ods_cust_src set address='100,xyz street, area,city' where id=1;
#melting of columns
create table learning.dwh_cust_tgt(id int,name varchar(100),address varchar(100));

```
#day1 load
truncate table learning.dwh_cust_tgt;
insert into learning.dwh_cust_tgt select id,name,address from learning.ods_cust_src;
select * from learning.dwh_cust_tgt;

#Source send the data in an incremental fashion (only inserted and updated), not complete
fashion
```
**#SCD type1 dim load**
```
truncate table learning.ods_cust_src;
truncate table learning.dwh_cust_tgt;
#Source send the data in an incremental fashion (only inserted and updated), not complete
fashion
insert into learning.ods_cust_src (id,name,address) values(1,'sojiv','1,xyz street,
area,city');#day1 (10 customers)
insert into learning.ods_cust_src (id,name,address) values(2,'rizwan','2,abc street,
area,city');#day2 (20 customers)
insert into learning.ods_cust_src (id,name,address) values(3,'sujithra','2,abc street,
area,city');#day3
update learning.ods_cust_src set address='100,xyz street, area,city' where id=1;#day3
#melting of columns
create table learning.dwh_cust_tgt(id int,name varchar(100),address varchar(100));
#day1 load
insert into learning.dwh_cust_tgt select id,name,address from learning.ods_cust_src;
select * from learning.dwh_cust_tgt;

#If the source send data in an incremental fashion
#select s.*,t.* from learning.ods_cust_src s
insert into learning.dwh_cust_tgt
select s.id,s.name,s.address from learning.ods_cust_src s
left join learning.dwh_cust_tgt t #left join
on t.id=s.id
where t.id is null;

update learning.dwh_cust_tgt t
#select s.*,t.* from
join learning.ods_cust_src s #join
on t.id=s.id
set t.address=s.address, s.name=t.name
WHERE t.address<>s.address or t.name<>s.name;
```

22. Task 22: Loading SCD Type2 table

Try to understand the concepts of SCD type2 load (first by using the examples given below with

some small table, then try the actual usecases provided in the portal)

#Source send the data in an incremental fashion (only inserted and updated), not complete fashion

**#SCD type2 dim load**

```
truncate table learning.ods_cust_src;
truncate table learning.dwh_cust_tgt;
#Source send the data in an incremental fashion (only inserted and updated), not complete fashion
insert into learning.ods_cust_src (id,name,address) values(1,'sojiv','1,xyz street, area,city');#day1 (10 customers)
insert into learning.ods_cust_src (id,name,address) values(2,'rizwan','2,abc street, area,city');#day2 (20 customers)
insert into learning.ods_cust_src (id,name,address) values(3,'sujithra','2,abc street, area,city');#day3
update learning.ods_cust_src set address='100,xyz street, area,city' where id=1;#day3
update learning.ods_cust_src set address='200,abc street, area,city' where id=2;#day3
#melting of columns
create table learning.dwh_cust_tgt(id int,name varchar(100),address varchar(100));
create table learning.dwh_cust_scd2_tgt(id int,name varchar(100),address varchar(100),start_date DATE,end_date DATE,
is_current TINYINT);

select * from learning.ods_cust_src;
select * from learning.dwh_cust_scd2_tgt;
#day2 load
update learning.dwh_cust_scd2_tgt t
#select s.*,t.* from
join learning.ods_cust_src s
on t.id=s.id
set t.end_date='2026-08-29',is_current=0
where s.address<>t.address
or s.name<>t.name;

insert into learning.dwh_cust_scd2_tgt
select s.id,s.name,s.address,current_date,null,1
from learning.ods_cust_src s
left outer join learning.dwh_cust_scd2_tgt t
on s.id=t.id
and t.is_current=1
where t.id is null#suchithra
or s.address<>t.address#sojiv data
or s.name<>t.name;
```

#day2 load
insert into learning.ods_cust_src (id,name,address) values(3,'sujithra','2,abc street, area,city');#day2
update learning.ods_cust_src set address='100,xyz street, area,city' where id=1;#day2

***#Try the same above update & insert queries..***

select * from learning.dwh_cust_scd2_tgt;

23. Task 23: Loading Fact Tables and Data Mart tables.
Try simple transformation functions like substr, case etc., functions like given below.
select substr('inceptez',6,3),case when 1=1 then 'one' else 'two' end;

***Implement the Datawarehouse fact tables such as fact_loans and fact_loan_summary tables following the portal.***

24. Task 24: Complete the Data mart load task by loading rest of the fact tables.
Fact_creditcards, fact_payments, fact_transactions, agg_branch_trans_summary & fact_high_value_loans

25. Task 25: Go through the **'Project Learnings'** section under **'BFSI Legacy DW Project1'** and try to realize whether we are able to catch up with the overall project implementation what we did so far.

## 2. PYTHON TASKS:

### A. Python is an indent based programming language

The following program throws an indentation error. Correct it and make sure it prints properly.

```
teams = ['Data', 'AI', 'DevOps']
for t in teams:
print('Hello', t, 'Team from Inceptez Technologies')
  print('Keep Learning and Exploring!')
```

### B. Commented line in Python

**Use Case 1:**

Add single-line and multi-line comments to describe what the below code does for Inceptez Technologies' training tracker.

```
students = 100
```

**trainers = 2**
**total = students + trainers**
**print(total)**

**Use Case 2:**
Convert the below block into a “dead code” using comments, then re-activate it later to print

**print("Welcome to Inceptez Python Learning")**

## C. Playing with Quotes

**Use Case 1:**
Create three string variables that correctly store and print:

**This is Inceptez's "Python" class for Data Engineers & AI Engineers**
→ Use single, double, and triple quotes appropriately.

**Use Case 2:**
Write a multiline string using triple quotes that prints:

**Welcome to Inceptez Technologies!**
**Python Training: Basics**
**Enjoy your learning journey.**

## D. Let's learn all about VARIABLES

**Use Case 1:**
Declare variables to store the following details:
- Student Name
- Course Name (e.g., “Python Fundamentals”)
- Training Institute Name (Inceptez Technologies)

Then print a formatted message:

Name: Arun is learning the course Python Fundamentals at the institute Inceptez Technologies

**Use Case 2:**
Demonstrate dynamic inference, dynamic typing using with fee by applying .18 gst and prove strongly typing character also by operating it with Eighteen percent gst

fee = 45000

## E. Variables Naming Conventions

**Use Case 1:**
Identify which variable names below are invalid for Inceptez's student database:

1) 2student = 'Ravi'
2) _student_id = 1001
3) studentName = 'Priya'
4) class name = 'Python'
5) inceptez_batch = 'Morning'

**Use Case 2:**
Declare 3 variables following naming styles for Inceptez projects:

PascalCase: DataEngineeringBatch
camelCase: dataEngineeringBatch
snake_case: data_engineering_batch

## F. Type identification & Casting

**Use Case 1:**
Write a program that asks for an employee's age.

1. Checks its type is of string (think about using isinstance() function)
2. Converts it to int (continue writing your program from here..)
3. Prints the years pending for retirement, for eg. 60 is the retirement age.

***Example:***
Enter your age: 40
You will retire in 20 years at Inceptez Technologies.

**Use Case 2 (Debug):**
Fix the type error in the following code for salary calculation:

```
salary = '50000'
bonus = 10000
print('Total Salary in Inceptez:', salary + bonus)
```

## G. Data types and casting

**Use Case 1 — Employee Salary Breakdown Using Numeric & String Types**

Employee Salary Breakdown

a. Write a program that asks the user for:

- employee_name (string)
- base_salary (float)
- hra_percent (integer)
- bonus_amount (float)

B. Convert inputs to the correct datatype if required.

Calculate:
HRA = base_salary * (hra_percent / 100)
Total Salary = base_salary + HRA + bonus_amount

C. Print the output like this:

**Employee: Arun**

**Base Salary: 40000.0**

**HRA @ 20%: 8000.0**

**Bonus: 5000.0**

Total Salary Payable: ₹53000.0

**Use Case 2: Student Result Classification**

a. Write a program that takes marks as input (initially as a string).
B. Check if the value can be converted to float.

C. Then classify (try using if condition with the help of AI, however we will learn about if condition soon):

Marks >= 90 --> Outstanding
Marks >= 75 --> Excellent
Marks >= 50 --> Pass
Marks < 50 --> Fail

D. If the input is not numeric, print:
Invalid marks entered — Please provide numeric input.

**Use Case 3: Bug Fixing — Datatype Mismatch**

The below code is intended to calculate total price, but it has datatype errors. Fix it.

*Incorrect code:*

```
item_name = input("Enter product name: ")
price = input("Enter price per item: ")
quantity = input("Enter quantity: ")

total_cost = price * quantity

print("You purchased " + quantity + " units of " + item_name)
print("Total payable: " + total_cost)
```

Expected output after fixing:

```
Enter product name: Notepad
Enter price per item: 35.50
Enter quantity: 3

You purchased 3 units of Notepad
Total payable: 106.5 INR
```

## H. Python Operators Usecases

**Use Case 1: Internet Data Usage Calculator**

Write a program that asks the user for:

- Total monthly data limit (in GB)
- Data used so far (in GB)

***Calculate using arithmetic operators:***

Remaining data = limit - used
Usage percentage = (used / limit) * 100

***Print:***

- Remaining data
- Usage percentage rounded to 2 decimals

If usage percentage is greater than or equal to 80, print:
"Warning: High usage, consider upgrading your plan."

**Use Case 2: Shopping Discount Calculation**

Write a program that takes:

- Original price (float)
- Discount percent (int)

Using assignment and arithmetic operators, calculate:
Discount amount = (price * discount_percent) / 100
Final price = price - discount_amount

Print:
Original price, discount applied, and final payable amount.

---

**Use Case 3 (Bug Fixing): Logical and Comparison Operator Errors**

The following code should determine voting eligibility, but it contains operator mistakes. Fix it.

***Incorrect code:***

```
age = input("Enter age: ")
citizen = input("Are you an Indian citizen? (yes/no)")

if age > "18" and citizen = "yes":
print("Eligible to vote")
else:
print("Not eligible")
```

***Expected behavior:***

- Convert age to integer before comparison.
- Only print "Eligible to vote" if age is 18 or above AND citizen input is "yes" (case-insensitive).

## I. Conditional Structure

**Use Case 1: Banking Eligibility Check**

Write a program that asks the user for:

- Age
- Monthly income

Conditions:

- If age < 18: print "Not eligible for a bank account."
- If age >= 18 and income < 15000: print "Eligible for basic savings account."
- If age >= 18 and income between 15000 and 50000: print "Eligible for savings + salary account."
- If age >= 18 and income > 50000: print "Eligible for premium account."

**Use Case 2: Check room availability**

-Check room availability

- If available:
  - If guest is VIP
    → Offer complimentary upgrade
  - Else if member 5+ years
    → Offer discount
  - Else
    → Standard price
- Else:

→ Show: "No rooms available"

**Use Case 3 (Bug Fixing): Nested Condition Logic Issue**

Fix the following code so that it correctly determines whether the entered temperature indicates normal, fever, or high fever.

Incorrect code:

```
temp = input("Enter body temperature in Celsius: ")

if temp < "37":
 print("Normal temperature")
 elif temp > "37" and temp < "39":
 print("Fever")
 else
 print("High fever")
```

Expected behavior:

- Convert temperature to float before comparison.

- Conditions should print:
  Normal temperature (less than 37)
  Fever (between 37 and 39)
  High fever (39 and above)

## J. Looping Constructs

**Use Case 1: Table Generator**
Write a program that takes a number from the user and prints the multiplication table from 1 to 10 for that number.

Example:
If user enters 5, output should be:
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50

**Use Case 2: Sum of Even and Odd Numbers**

Write a program that asks the user for a positive integer `n`.
Using a loop, calculate and print:

- Sum of all even numbers from 1 to n
- Sum of all odd numbers from 1 to n

**Use Case 3 (Bug Fixing): Infinite Loop Issue**

Fix the code below so that it prints numbers from 1 to 10 and stops correctly.

Incorrect code:

```
i = 1
while i <= 10:
print(i)
```

Expected behavior:
The program must increment `i` and stop when 10 is printed.

## K. Collection Types

**Use Case 1: Product Price Lookup**

Create a dictionary with at least 5 products and their prices.

Ask the user to enter a product name.

If found, print the price.

If not found, print: "Product not available." (Hint: use dictionary_var.get(key) function)

**Use Case 2: City Entry and Duplicate Removal**

Ask the user to enter city names repeatedly.
Stop when the user types "exit".

Requirements:

- Store every entered city name in a **list** (even if it's repeated).
- Also store the cities in a **set** to maintain only unique values.

Finally print:

- The complete list of entered cities (with duplicates).
- The set of unique cities (duplicates removed).

**Use Case 3 (Bug Fixing): List Index Error**
Fix the following code so that it prints all items correctly without an index error:

Incorrect code:

```
items = ["Pen", "Book", "Mouse", "Keyboard"]
i = 0
while i <= len(items):
print(items[i])
i = i + 1
```

**Expected behavior:**
The loop should print all the items exactly once and exit without an error.

## L. Exception Handling

**Use Case 1: Division Safe Calculator**
Ask the user for two numbers.
Perform division and print the result.
If the user tries to divide by 0, print:
"Error: Division by zero is not allowed."

**Use Case 2: Safe Integer Input**

Ask the user to enter a number.

Try converting it to an integer.

If conversion fails, print:

"Invalid input. Please enter a numeric value."

**Use Case 3 (Bug Fixing): Multiple Exception Handling**
Fix the below code so it handles both invalid input and division by zero correctly.

Incorrect code:

```
num1 = int(input("Enter number 1: "))
num2 = int(input("Enter number 2: "))
result = num1 / num2
print("Result:", result)
```

Expected behavior:

- If user enters non-numeric values ⟶ print "Invalid input"
- If num2 is zero ⟶ print "Cannot divide by zero."
- Otherwise print the result.

## M. Function Based Programming (Important)

**Usecase 1: Food Delivery Discount (Convert this program as function with Exception Handling also added and try keeping this function in a pkg.subpkg.module & import inside a ipynb)**

**Use Case 2: Salary Processing System (HR / Payroll Automation)**

*Demonstrates input/output functions, default arguments, arbitrary args, returning values, and composing multiple functions.*

**What Problem It Solves:**

Companies calculate employee salary differently depending on bonus percentage, incentives, PF, tax, etc.

**How Function-Based Programming Helps:**

- We create multiple small reusable functions (bonus calc, tax calc, net salary calc)
- Then compose them together.

**Usecase3: Number Utility Tool**

Create functions:

- is_even(num) → returns True/False
- find_max(*numbers) → returns highest number from arguments
- perform_operation(num, operation) →
    - If operation = "square" → return num*num
    - If operation = "cube" → return num*num*num
    - If unknown → return "Invalid operation"

Test the functions with at least 5 numbers.

**Use Case 4 (Bug Fixing): Fix the Function Logic**

The following function is supposed to calculate the square of a number and return the result. However, it contains multiple errors. Fix the code so it works correctly.

```
def find_square():
    num = input("Enter a number: ")
    result = num * num
    print("Square of the number is" result)
    return result
```

***Expected behavior:***

- The input must be converted to an integer.
- The function should print the correct result using proper formatting.
- The function must return the square value without errors.

**Use Case 5: Invoice Generator (**kwargs)**

Create a function generate_invoice(**products).

- Each key is a product name and each value is its price.
- Print all product names with prices and also print the total amount.

**Use Case 6: Import vs from import**

Create simple functions like add(a,b) and div(a,b) that returns sum and division of 2 input.

- Create a package/sub package/generic_functions.py and place this function .

- Create another package/sub package/consumer.py and import the above 2 functions using import, from import with and without alias.

**Use Case 7: global vs local variable**

Create a functions add(a,b) that stores the result of a+b to variable c and make the variable c as global variable that should be accessible outside of the function after executing the function.